Politechnika Śląska

UCZELNIA BADAWCZA
INICJATYWA DOSKONAŁOŚCI

ASCII ART
SZTUKA TWORZONA ZA POMOCĄ ZNAKÓW KODU ASCII

Błażej Jamrozik

”Piękno nie jest właściwością samych rzeczy. Istnieje w umyśle, który je ogląda, a każdy umysł postrzega inne piękno.”

David Hume

Agenda

01
Wprowadzenie i cel
Definicja problemu i założeń projektu.

02
Algorytm działania
Szczegółowe wyjaśnienie konwersji.

03
Aplikacja: interfejs i funkcje
Przegląd kluczowych elementów UI.

04
Prezentacja działania
Demonstracja programu w praktyce.

05
Analiza wydajności
Pomiary czasowe i skalowanie rozwiązania.

06
Podsumowanie i wnioski
Główne obserwacje.

Cele aplikacji

Wczytywanie danych

Obsługa pojedynczych obrazów oraz całych folderów.

Transformacja pikseli

Przekształcanie pikseli na znaki ASCII bazując na jasności.

Zapis wyników

Możliwość zapisu jako plik .txt oraz .png

Wybór implementacji

Przełączanie między DLL w wersji CPP oraz ASM, zarządzanie wielowątkowością.

PO CO TAKA APLIKACJA?

Motywacje projektu

Alternatywna wizualizacja

ASCII Art jako unikalna forma przekazu obrazu bez grafiki.

Platforma badawcza

Doskonały obszar do badania przetwarzania danych, optymalizacji, SIMD i wielowątkowości.

Porównanie wydajności

Możliwość analizy efektywności różnych implementacji.

Oczekiwane efekty

01 **Aplikacja z GUI**
Intuicyjny interfejs użytkownika

02 **Konwersja wielu plików**
Obsługa pojedynczych obrazów i folderów.

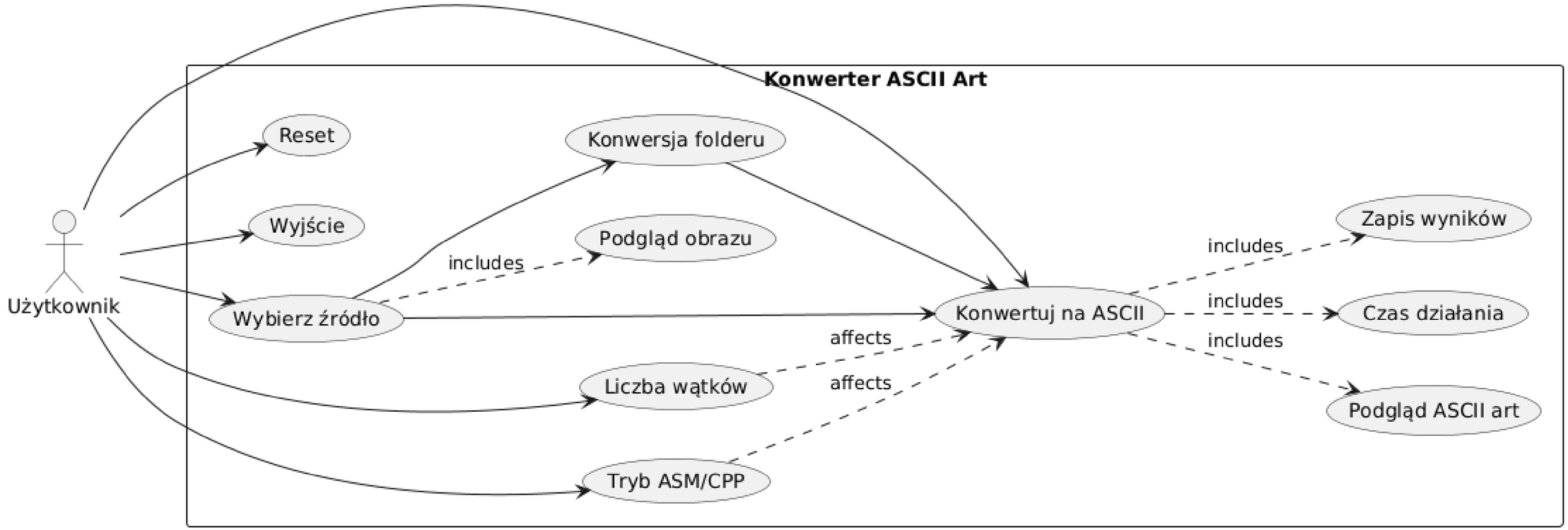
03 **Wybór trybu pracy**
Dynamiczne przełączanie implementacji algorytmu.

04 **Pomiar czasu działania**
Precyzyjne metryki dla DLL i całego programu.

05 **Raportowanie wyników**
Wizualizacja danych

06 **Zaliczenie projektu**
Uzyskanie oceny pozytywnej.

Uproszczony diagram przypadków użycia



Podstawy algorytmu

Każdy piksel jest opisywany jedną liczbą — jego jasnością. Następnie jasność mapujemy na przestrzeń znaków, tworząc tekstową reprezentację wizualną.

Wejście: Obraz rastrowy

Format (np. JPG, PNG). Obrazy kolorowe są najpierw konwertowane na odcienie szarości.

Etap 1: Konwersja do jasności

Każdy piksel jest przekształcany na wartość jasności w zakresie 0–255 (od czerni do bieli).

Etap 2: Wybór znaku ASCII

Wartość jasności piksela jest mapowana na odpowiedni znak z predefiniowanej tablicy ASCII.

Mapa jasności → ASCII

Od najbardziej „zapełnionych” wizualnie:

- `$`, `@`, `B`

Do najlżejszych:

- `,`, `^`, `spacja`

`Indeks = (jasność · 63) >> 8`

Jasność w zakresie 0–255 skalujemy do zakresu 0–63. Binarne przesunięcie w prawo o 8 bitów (`>> 8`) efektywnie zastępuje dzielenie przez 256, dzięki czemu mapowanie jest szybkie, minimalizując koszt obliczeniowy.

Tablica 64 znaków ASCII:

```
$@B%8&WM#*oahkbdpqwmZO0QLCJUYXzcvunxrjft/
()1{}[]?-_+~<>i!lI;:,^
```

Wektoryzacja SIMD z SSE4.1

Implementacja w Asemblerze wykorzystuje instrukcje SSE4.1 do efektywnego przetwarzania danych. Zamiast schematu "1 piksel = 1 operacja", osiągamy "16 pikseli = 1 operacja", co drastycznie zwiększa przepustowość i zmniejsza narzut instrukcyjny, czyniąc konwersję znacznie szybszą.

Kluczowe instrukcje

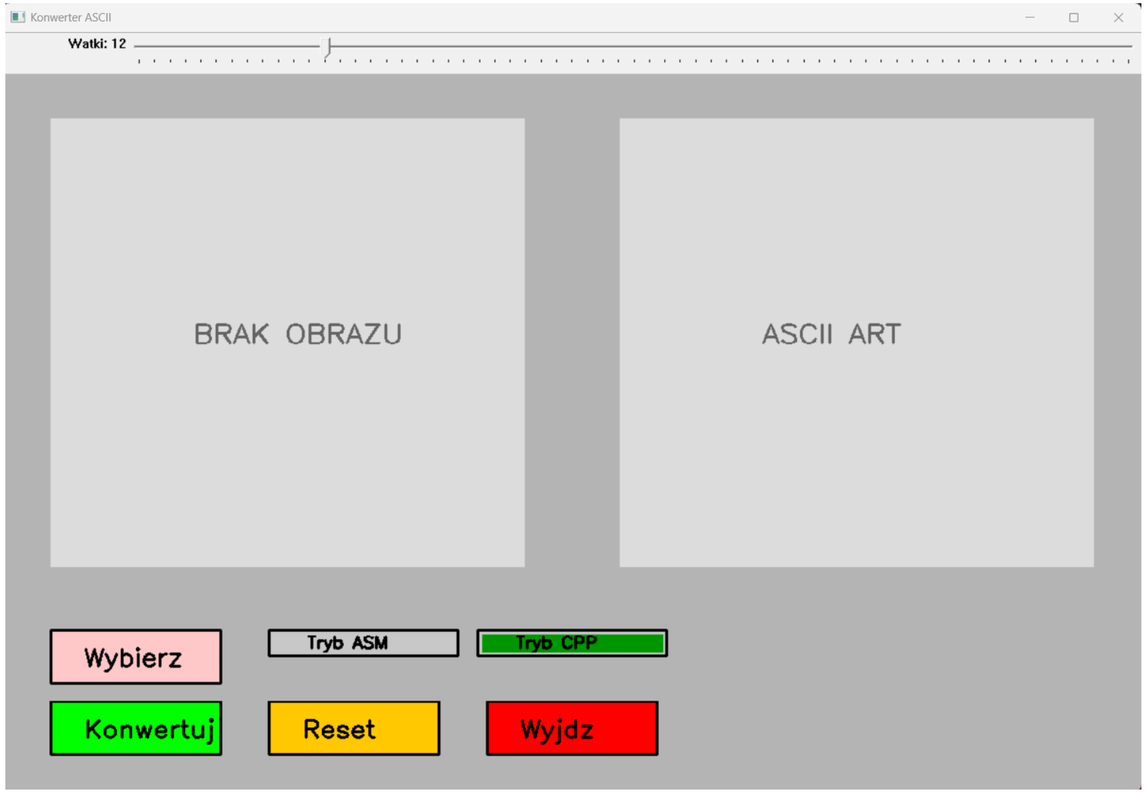
- `pmovzxbw`: rozszerzenie 8-bit do 16-bit.
- `pmullw`: mnożenie przez 63 (skalowanie).
- `psrlw`: dzielenie przez 256 (przesunięcie bitowe w prawo o 8 bitów).
- `packuswb`: scalanie do bajtów wynikowych.

Widok Główny GUI

Interfejs użytkownika prezentuje dwa kluczowe panele wizualne, umożliwiające natychmiastową ocenę efektów konwersji obrazu na ASCII.

- **Lewy panel:** Wyświetla oryginalny obraz źródłowy.
- **Prawy panel:** Przedstawia przetworzony wynik w formie grafiki ASCII.

Dodatkowo, sekcja komunikatów dostarcza informacje o postępie operacji, wybranym trybie przetwarzania oraz ewentualnych błędach wejścia/formatu. Dzięki temu użytkownik może na bieżąco monitorować proces i oceniać jakość konwersji.



Kontrolki Użytkownika i Konfiguracja

Aplikacja udostępnia zestaw intuicyjnych kontrolek, które pozwalają użytkownikowi na pełne zarządzanie procesem konwersji. Kluczowe jest określenie konfiguracji przed rozpoczęciem przetwarzania, co ma fundamentalne znaczenie dla dokładnych pomiarów czasu i wydajności.

1

Akcje Główne

- **Wybierz:** Umożliwia wybór pojedynczego pliku obrazu lub całego folderu
- **Konwertuj:** Rozpoczyna proces konwersji obrazu(ów) na ASCII.
- **Reset:** Czyści bieżący stan interfejsu GUI, przygotowując aplikację do nowego zadania.
- **Wyjdź:** Zamyka aplikację.

2

Tryby Pracy

Wybór trybu wpływa na implementację algorytmu konwersji, co ma bezpośrednie przełożenie na wydajność:

- **ASM:** Zoptymalizowana wersja z użyciem asemblera.
- **CPP:** Standardowa implementacja w C++.

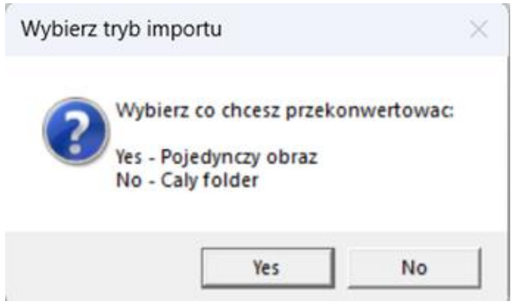
3

Liczba Wątków

Suwak pozwala na dynamiczne dostosowanie liczby wątków roboczych, w zakresie od **1 do 64**. Domyślnie aplikacja ustawia wartość `hardware_concurrency`, optymalizując wykorzystanie dostępnych zasobów sprzętowych. Pozwala to na precyzyjne badanie wpływu równoległości na wydajność.

Tryby Działania Aplikacji

Aplikacja oferuje dwa główne tryby działania, dostosowane do różnych scenariuszy użytkowania i analizy wydajności.



Tryb Pojedynczego Obrazu

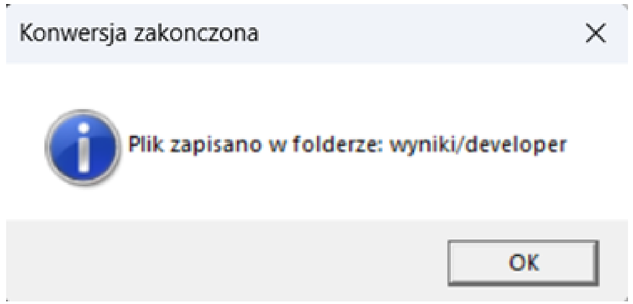
Idealny do szybkiej konwersji i inspekcji pojedynczych plików graficznych.

- Konwersja jednego pliku obrazu.
- Wynik konwersji jest wyświetlany bezpośrednio w GUI.
- Prezentuje szczegółowe czasy:
  - całkowity czas przetwarzania
  - czas działania biblioteki DLL.
- ASCII Art jest zapisywane do plików .txt oraz .png.

Tryb Folderu

Przeznaczony do przetwarzania dużych zbiorów danych, umożliwiając analizę skalowalności i wydajności.

- Konwersja wielu plików obrazów jednocześnie.
- Jeden przykładowy wynik konwersji jest wyświetlany bezpośrednio w GUI.
- Wyniki są automatycznie zapisywane do dedykowanych katalogów.
- Generowany jest raport w formacie CSV zawierający kompleksowe statystyki przetwarzania.
- Umożliwia przegląd wyników i dogłębną analizę statystyczną.



Prezentacja Działania

Poniżej przedstawiono kroki demonstracyjne procesu konwersji pojedynczego obrazu, ilustrujące łatwość obsługi i natychmiastowe efekty wizualne.

1. Wybór Obrazu

Użytkownik wybiera plik graficzny za pomocą przycisku "Wybierz".

2. Konfiguracja Trybu i Wątków

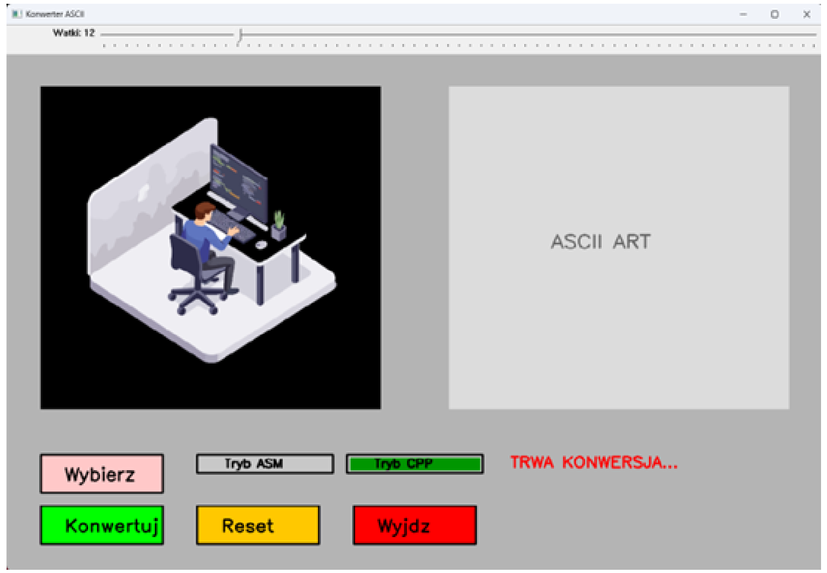
Następnie wybiera preferowany tryb pracy (ASM, CPP) oraz optymalną liczbę wątków dla danego zadania.

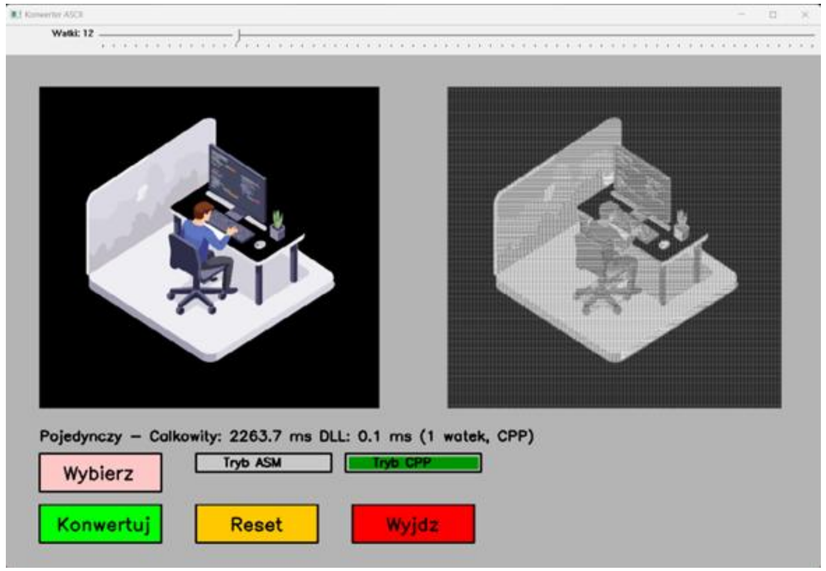
3. Uruchomienie Konwersji

Kliknięcie przycisku "Konwertuj" rozpoczyna przetwarzanie obrazu.

4. Analiza Wyników ASCII

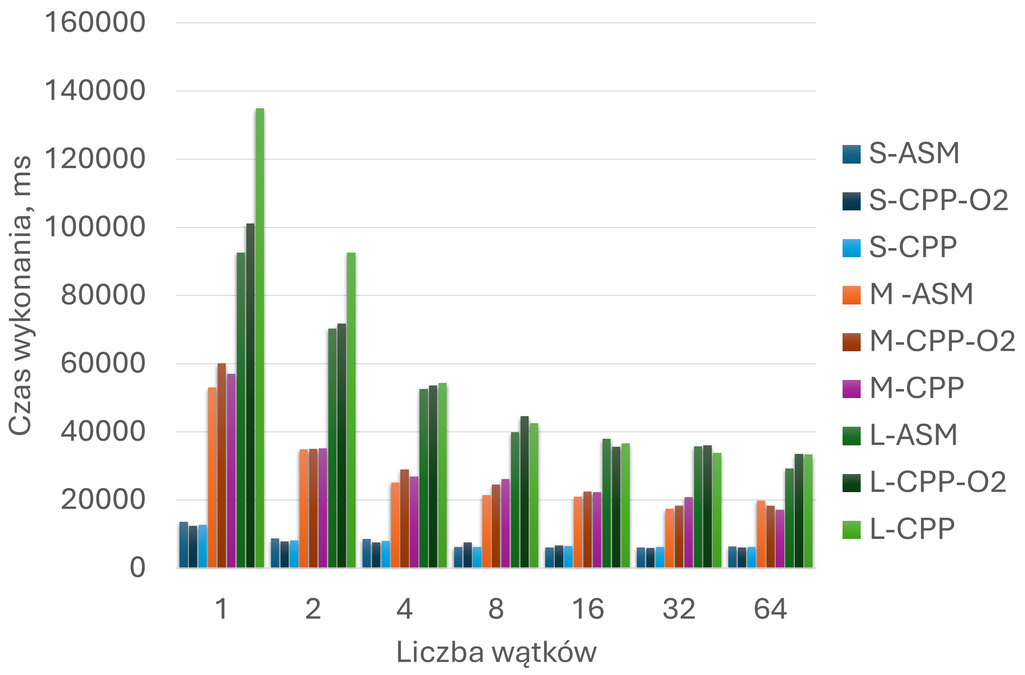
Po zakończeniu konwersji, wynik ASCII jest natychmiast widoczny w prawym panelu GUI.





Analiza Czasowa: Czas Całkowity

Szczegółowa analiza całkowitego czasu przetwarzania ujawnia kluczowe wzorce wydajnościowe w zależności od liczby używanych wątków.



Wykres zależności średnich czasów wykonania całego programu dla kolekcji danych S, M, L od liczby wątków z podziałem na wersje DLL

Zauważany Zysk

Zauważalny jest skok wydajności przy przejściu z **1 na 2 wątki**, co świadczy o efektywnym wykorzystaniu równoległości na początkowym etapie.

Stabilne Optimum

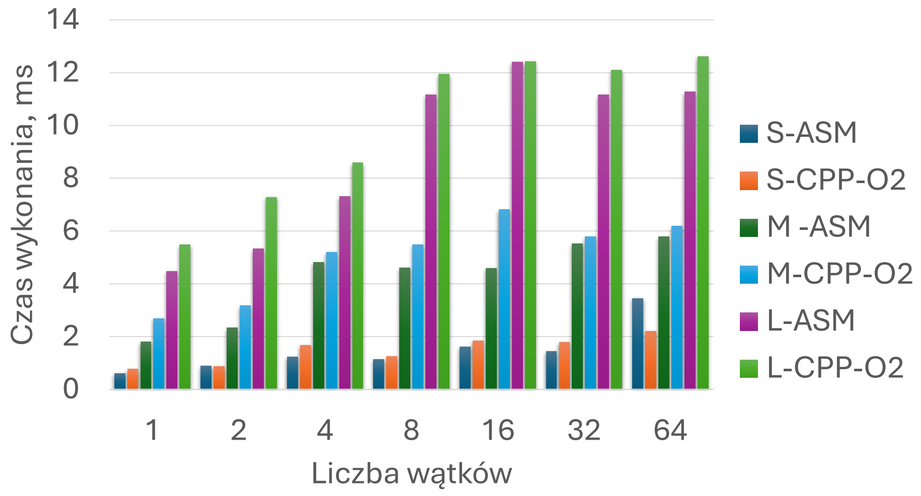
Najlepsza równowaga między przyspieszeniem a narzutem osiągana jest w przedziale **8–16 wątków**.

Narzuty i Koszty

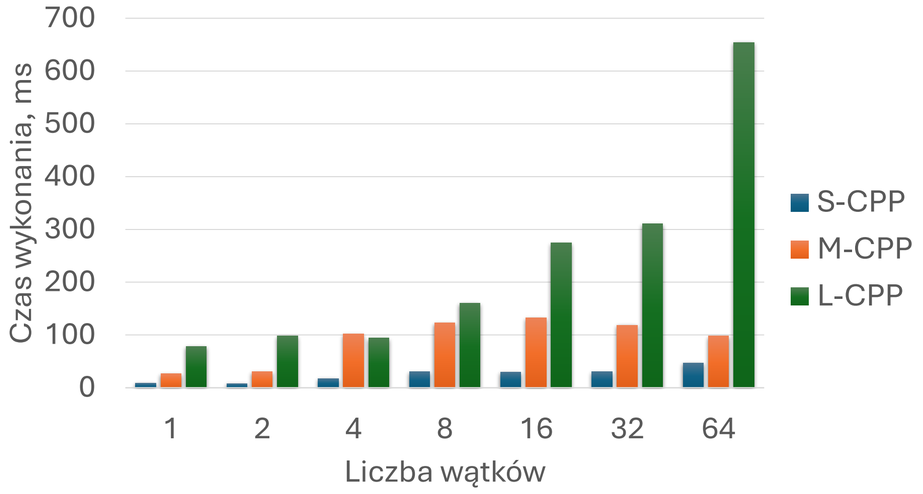
Powyżej optimum, koszty związane z synchronizacją, I/O oraz zarządzaniem wątkami zaczynają dominować nad zyskami z równoległości.

Analiza Czasowa: Czas Działania DLL

Analiza czasu spędzonego w bibliotece DLL (Dynamic Link Library), odpowiedzialnej za sam algorytm konwersji, dostarcza cennych informacji na temat wydajności poszczególnych implementacji.



Wykres zależności średnich czasów wykonania DLL w wersji ASM oraz CPP O2 dla kolekcji danych S, M, L od liczby wątków



Wykres zależności średnich czasów wykonania DLL w wersji CPP dla kolekcji danych S, M, L od liczby wątków

Czas DLL na obraz

Wzrasta proporcjonalnie do liczby wątków, gdyż DLL działa jednowątkowo.

ASM vs C++ O2

Implementacje te osiągają zbliżone czasy, demonstrując efektywność nowoczesnych kompilatorów.

C++ bez optymalizacji

Jest **10–15 razy wolniejszy** niż jego zoptymalizowane odpowiedniki, podkreślając znaczenie optymalizacji kompilatora.

Mimo że zewnętrzna wielowątkowość przyspiesza przetwarzania, nie zmienia ona fundamentalnego kosztu pojedynczego wywołania konwertera wewnątrz biblioteki DLL.

ASM vs C++ O2: Kluczowe Przewagi Wydajnościowe

Dogłębna analiza kluczowych przewag wydajnościowych Assemblera nad C++ w optymalizacji O2.

Mniej instrukcji

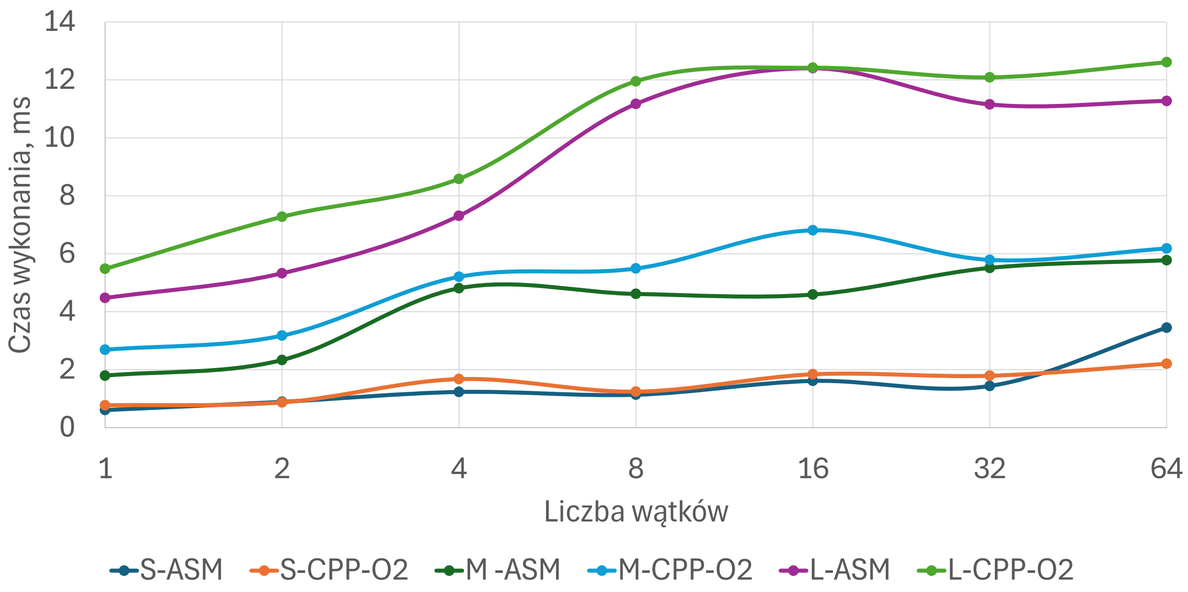
Bezpośrednia kontrola nad procesorem pozwala na minimalizację liczby wykonywanych operacji.

Lepsze wykorzystanie SIMD

Efektywne wykorzystanie instrukcji SIMD (Single Instruction, Multiple Data) do przetwarzania równoległego.

Bezpośredni dostęp do pamięci

Możliwość bezpośredniego manipulowania pamięcią, omijając abstrakcje wysokopoziomowe.



Wykres zależności średnich czasów wykonania DLL w wersji ASM oraz CPP O2 dla kolekcji danych S, M, L od liczby wątków

ASM często oferuje znaczące przyspieszenie w krytycznych sekcjach kodu, co jest kluczowe w zastosowaniach wymagających ekstremalnej wydajności.

Podsumowanie Kluczowych Wniosków

Projekt efektywnie połączył teorię, programowanie systemowe i empiryczne badania wydajności, dostarczając wymiernych rezultatów.

Algorytm Mapowania Stałoprzecinkowego

Zastosowanie algorytmu opartego na stałoprzecinkowym mapowaniu kolorów do znaków ASCII zapewnia **szybkość i stabilność** procesu konwersji, minimalizując błędy i zapewniając przewidywalne wyniki.

Wydajność Assemblera SIMD

Implementacja w **Assemblerze z instrukcjami SIMD** (Single Instruction, Multiple Data) gwarantuje największą wydajność, szczególnie przy dużym obciążeniu procesora, wykorzystując pełen potencjał sprzętu.

C++ O2 vs. ASM

Nowoczesne kompilatory C++ z optymalizacją -O2 pozwalają na osiągnięcie wydajności **bardzo zbliżonej do kodu asemblerowego**, co świadczy o postępie w technologiach kompilacji.

Rola Wielowątkowości

Wielowątkowość jest najbardziej efektywna przy przetwarzaniu wielu obrazów jednocześnie, znacząco redukując ogólny czas wykonania dla zadań wsadowych. Dla pojedynczego obrazu jej wpływ jest ograniczony.

Kierunki Dalszego Rozwoju

Projekt ASCII Art Converter posiada szerokie możliwości dalszego rozwoju, otwierając drzwi do innowacyjnych zastosowań i rozszerzeń funkcjonalności.

ASCII Kolorowe / HTML / SVG

Rozszerzenie formatów wyjściowych o kolorowe ASCII, renderowanie w **HTML** z zachowaniem stylów lub generowanie skalowalnej grafiki wektorowej **SVG**.

ASCII to nie tylko zabawa z pojedynczym obrazem. Projekt może ewoluować w kierunku zaawansowanych narzędzi artystycznych.

Automatyczny Dobór Palety

Implementacja algorytmów inteligentnego doboru palety znaków i kolorów w zależności od charakterystyki obrazu źródłowego, aby uzyskać **optymalny efekt wizualny**.

Konwersja Animacji

Możliwość przetwarzania serii obrazów na **ASCII Video**, otwierając drogę do tworzenia animowanych dzieł sztuki.

Dziękuję za uwagę!